



It's Your Life

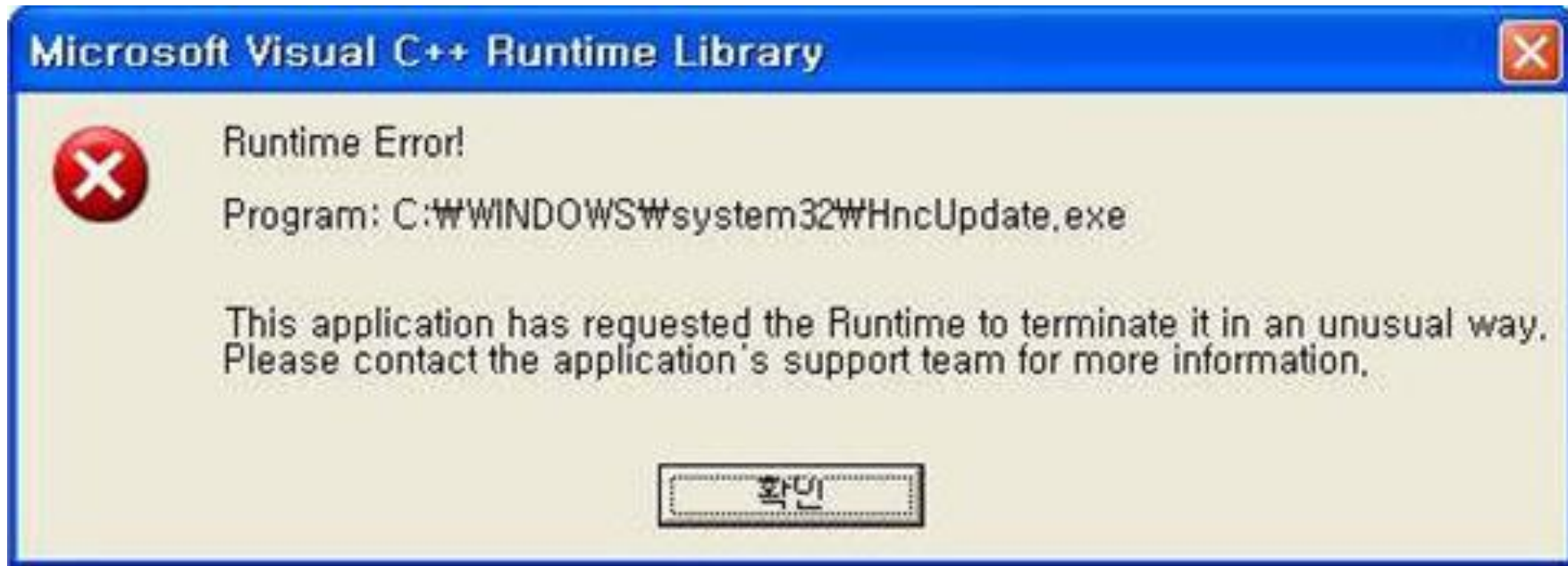
with





Try/Catch

Error 에 대해서 알아봅시다!





Error 에 대해서 알아봅시다!

- Error는 프로그램이 더 이상 실행이 불가능하게 만드는 예상치 못한 큰 문제입니다!
- 즉, Error 가 발생하면 프로그램이 죽는다(Die) / 깨진다(Crush) 라는 표현을 사용할 정도로 위험한 녀석입니다
- 그럼 별도의 처리가 필요하겠죠!?



이런 코드가 있다고 가정해 봅시다!

```
function program() {  
  const random = Math.random();  
  if (random > 0.5) {  
    // Error 가 예상되는 작업  
    const someResult = doSomething();  
  
    if (someResult === 'Error') {  
      alert('404, 에러 발생!! 비상!!!!!!!!!!');  
    }  
  }  
}  
  
program();
```

에러가 예상되는 작업인
doSomething() 을 실행할 때
에러가 발생한다면!?

다행 스럽게 결과 값이 Error 로
나온다면 이렇게 Error 처리
코드를 넣어서 해결하면 되겠죠?

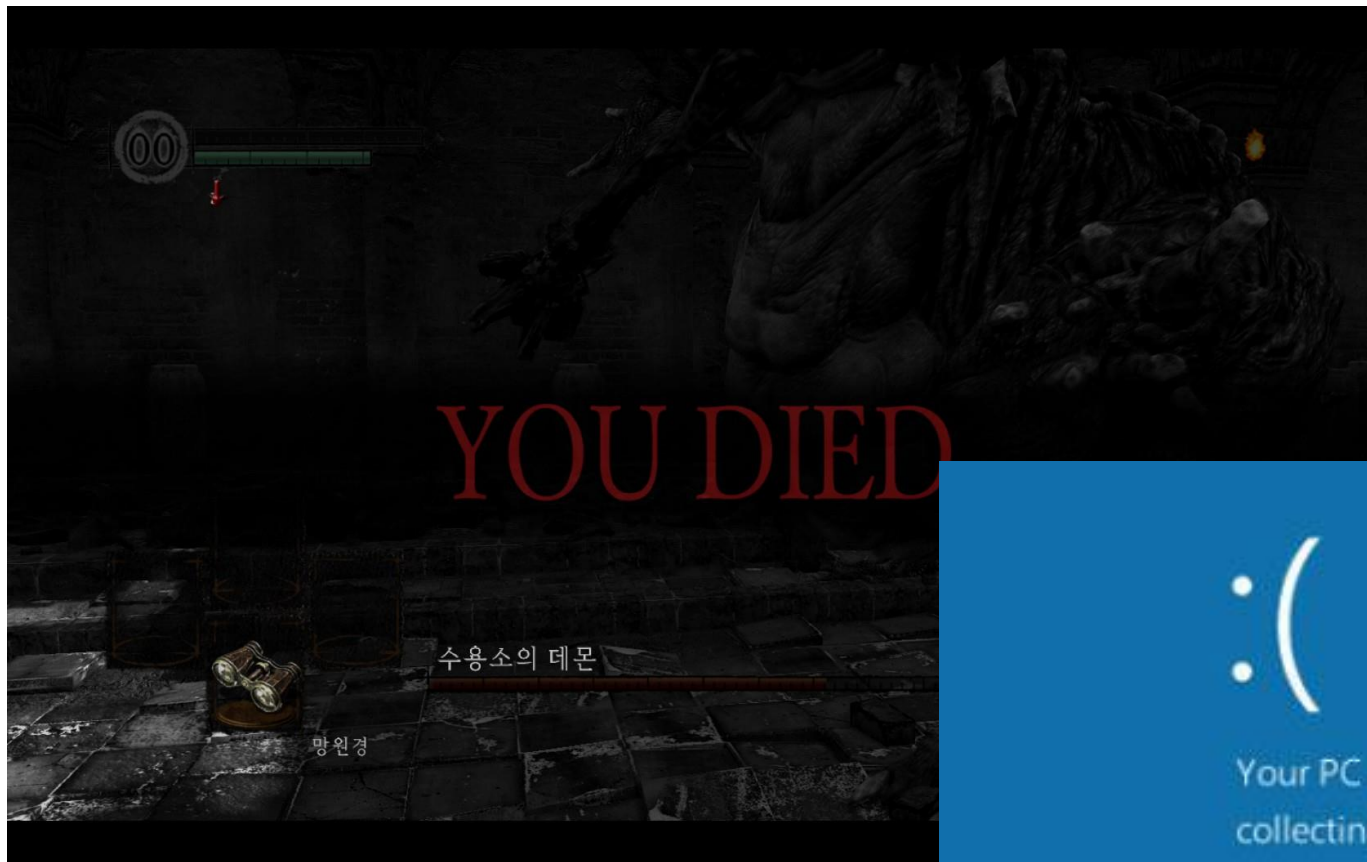
그런데 말입니다



에러가 예상되는 작업이
1000000 개라면!?

전부 에러에 대한
처리를 해줘야 할까요? ☺

그런데 말입니다...
처음에 말씀 드렸듯
Error 는 예상치 못한 에러이기
때문에 프로그램이 죽어 버립니다!



Your PC ran into a problem and needs to restart. We're just collecting some error info, and then we'll restart for you.

0% complete



For more information about this issue and possible fixes, visit
<http://windows.com/stopcode>

If you call a support person, give them this info:
Stop code: MANUALLY_INITIATED_CRASH

그래서 말입니다



```
promise
  .then((result) => {
    console.log('무사 통과! 생성된 숫자 :', result);
  })
  .catch((error) => {
    console.log('Error 발생! 생성된 숫자 :', error);
  });
```

예상치 못한 에러가 발생할
가능성이 있는
비동기 작업을 할 땐!

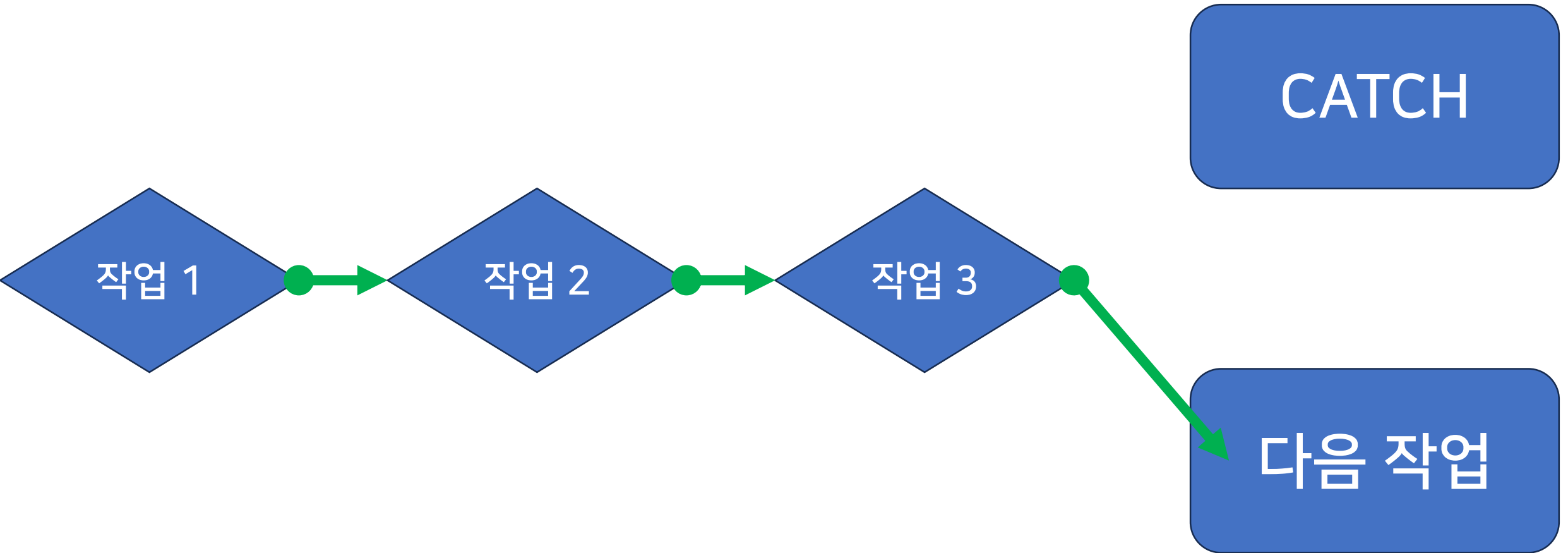
이렇게 JS 가 문법적으로
catch 같은 기능을 제공 합니다!



무사히 작업 완료



작업 중 에러 발생

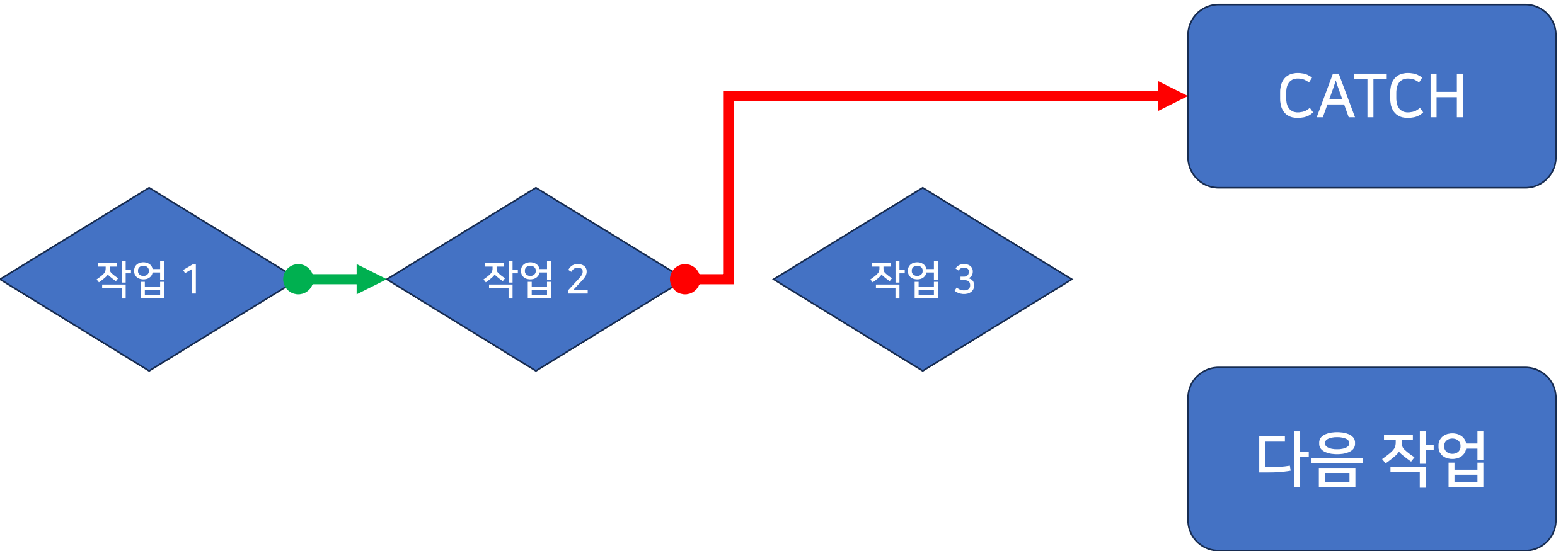




무사히 작업 완료



작업 중 에러 발생

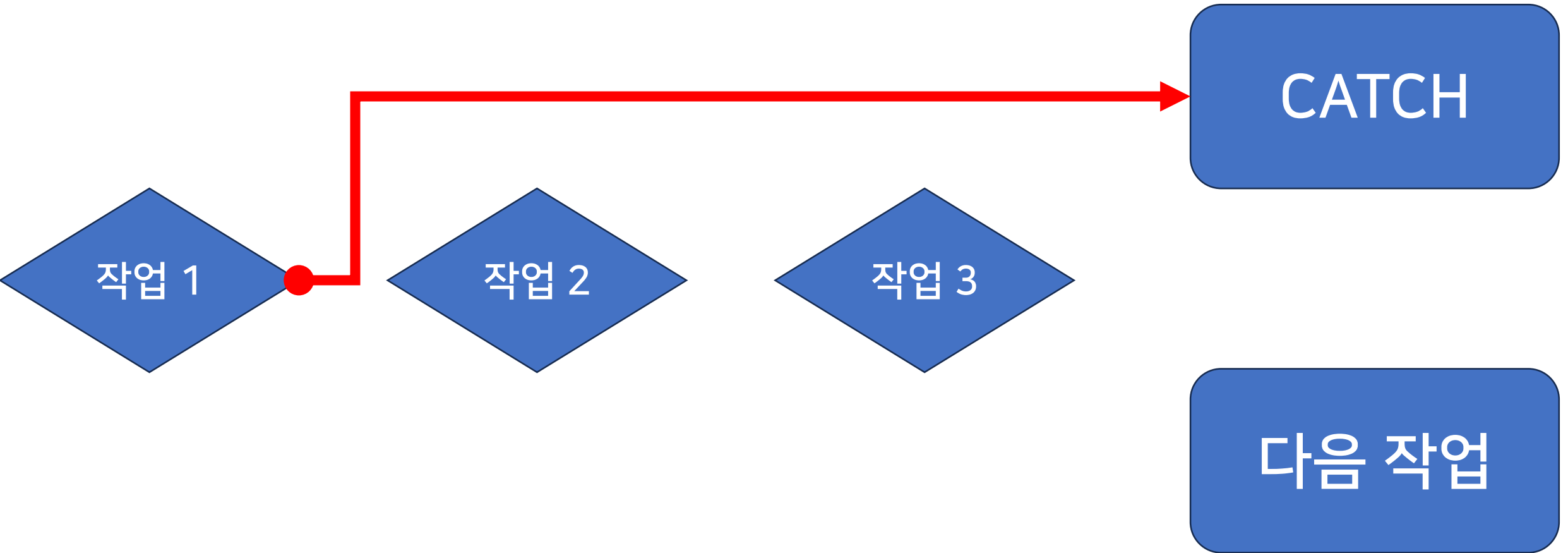




무사히 작업 완료



작업 중 에러 발생

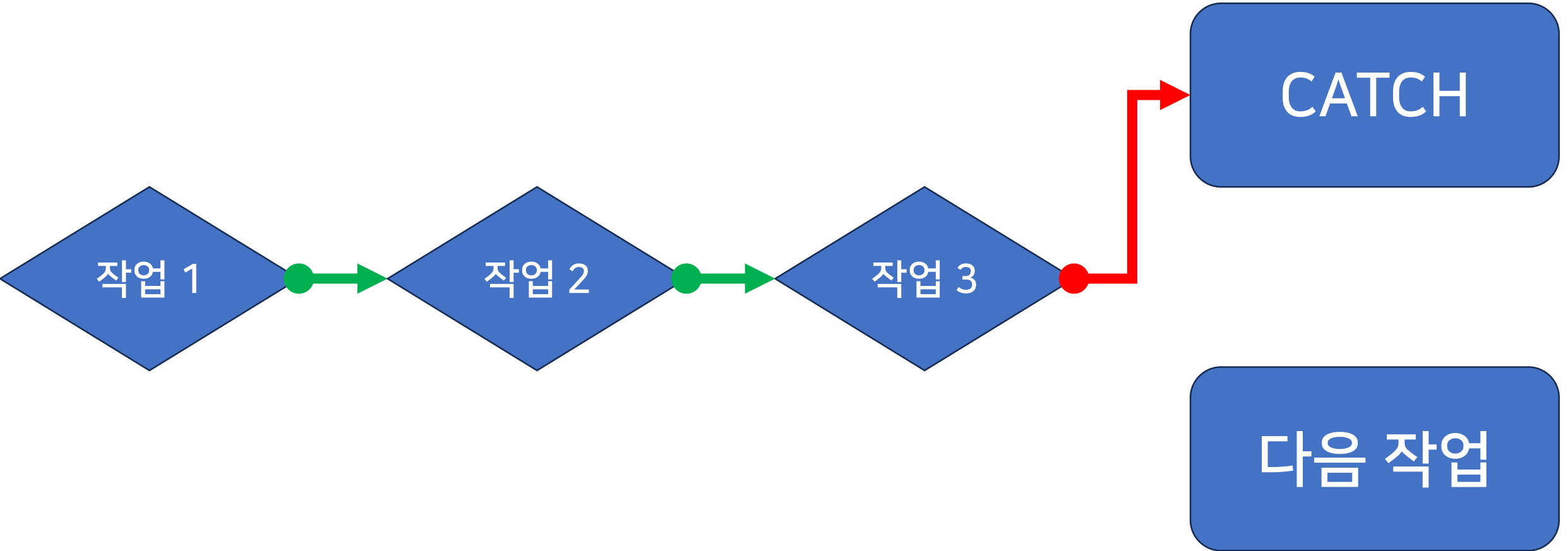




무사히 작업 완료



작업 중 에러 발생



그래서 말입니다



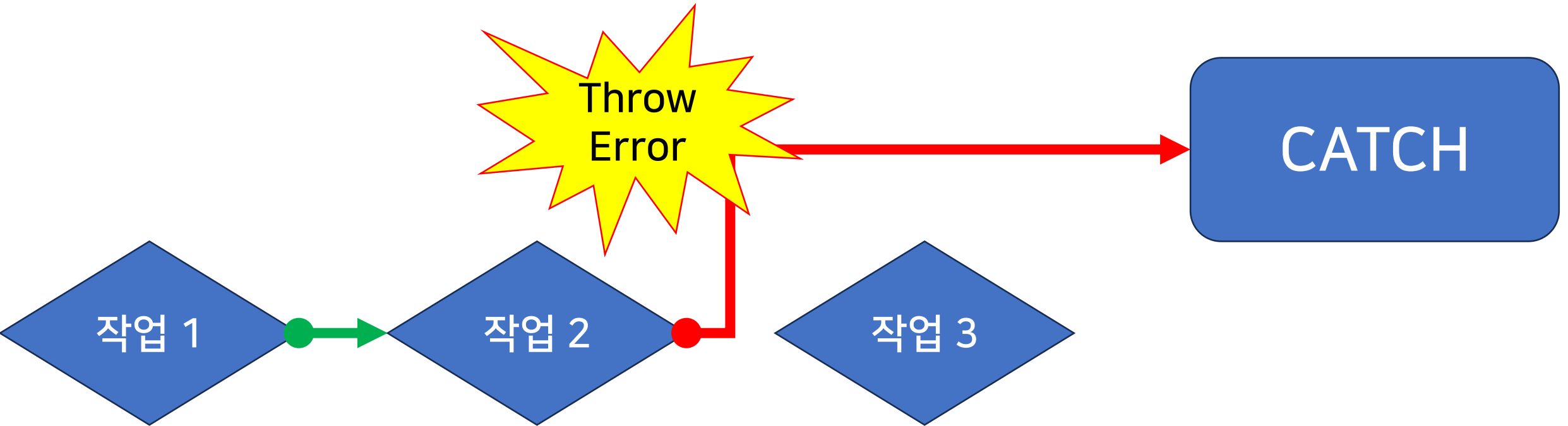
- 하나의 catch 구문을 사용하여 작업에서 예상되는 모든 Error 를 받아 줍니다!
- 그리고, catch 구문이 있을 경우 프로그램은 죽지 않고 유지가 됩니다!!
- 그리고 Error 가 발생하면 기존의 프로그램의 실행 흐름에서 벗어나 catch 구문으로 Error 를 던져주기 때문에 에러를 발생 시킬 때는 throw 키워드를 씁니다!



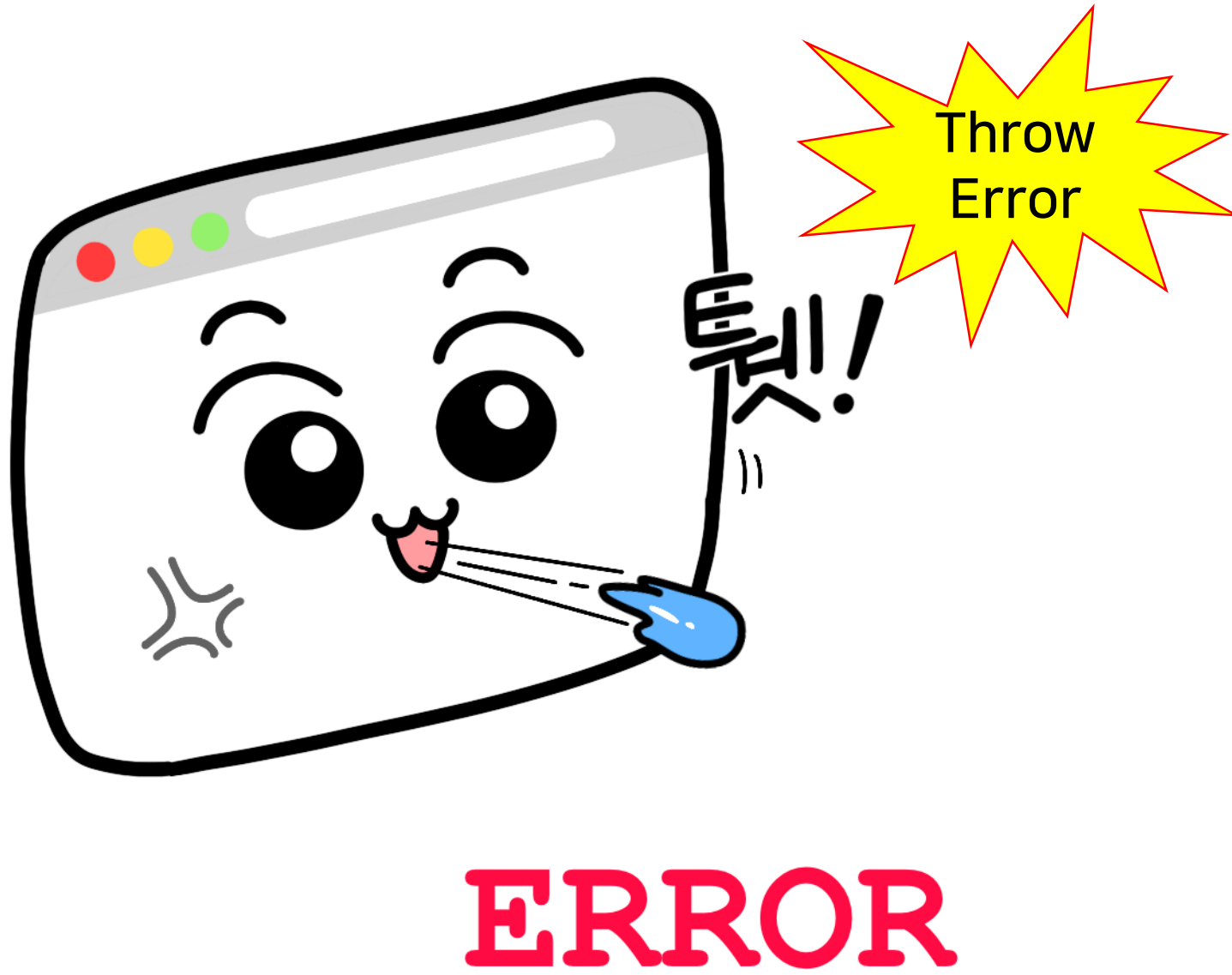
무사히 작업 완료



작업 중 에러 발생







CATCH



그래서 말입니다



```
throw new Error('망했어요...');
```

에러 던지기

새로운 Error
객체 생성

Error 메시지
전달

그런데 말입니다



.catch 구문 말고는
Catch 를 쓸 수 없나요!?

Try / Catch



에러가 예상되는 작업 어디서나!



- Try/Catch 구문은 에러가 예상되는 작업에서는 어디서든 사용 가능!!

```
try {  
    // 에러가 예상되는 작업 코드  
} catch (error) {  
    // 에러 처리 코드  
}
```

에러 발생시 사용되는
reject 로 전달된
에러 정보는
요기 error 로 받습니다!

프로그램이 어찌 굴러갈지 예상이 되시나요!?



```
setInterval(() => {  
  const rand = parseInt(Math.random() * 10);  
  console.log(`생성된 수는 ${rand}`);  
  
  if (rand > 5) return console.log('아싸 로또 사자!');  
  
  throw new Error('망했어요...');  
}, 2000);
```

2초마다 한번씩
랜덤 숫자를 생성해서

해당 숫자가 5 이상이면
콘솔 출력을

5 미만이면
에러를 던지는 코드



```
setInterval(() => {  
  const rand = parseInt(Math.random() * 10);  
  console.log(`생성된 수는 ${rand}`);  
  
  if (rand > 5) return console.log('아싸 로또 사자!');  
  
  throw new Error('망했어요...');  
}, 2000);
```

생성된 수는 0

d:\git\KB_FE\Vue\chapter02\03_try_catch.js:5

```
  throw new Error('망했어요...');
```

^

Error: 망했어요...

at Timeout._onTimeout (d:\git\KB_FE\Vue\chapter02\03_try_catch.js:5:9)

at listOnTimeout (node:internal/timers:569:17)

at process.processTimers (node:internal/timers:512:7)

Node.js v18.17.0

[Done] exited with code=1 in 2.111 seconds



생성된 수는 0

```
d:\git\KB_FE\Vue\chapter02\03_try_catch.js:5
  throw new Error('망했어요...');
  ^
```

Error: 망했어요...

```
at Timeout._onTimeout (d:\git\KB_FE\Vue\chapter02\03_try_catch.js:5:9)
at listOnTimeout (node:internal/timers:569:17)
at process.processTimers (node:internal/timers:512:7)
```

Node.js v18.17.0

[Done] exited with code=1 in 2.111 seconds

에러가 발생하면
프로그램이 죽어버립니다 ☹️

Try / Catch



```
setInterval(() => {  
  try {  
    const rand = parseInt(Math.random() * 10);  
    console.log(`생성된 수는 ${rand}`);  
  
    if (rand > 5) return console.log('아싸 로또 사자!');  
  
    throw new Error('망했어요...');  
  } catch (error) {  
    console.log(error);  
  }  
}, 2000);
```





생성된 수는 0

Error: 망했어요..

```
at Timeout._onTimeout (d:\git\KB_FE\Vue\chapter02\03_try_catch.js:8:11)
```

```
at listOnTimeout (node:internal/timers:569:17)
```

```
at process.processTimers (node:internal/timers:512:7)
```

생성된 수는 9

아싸 로또 사자!

생성된 수는 3

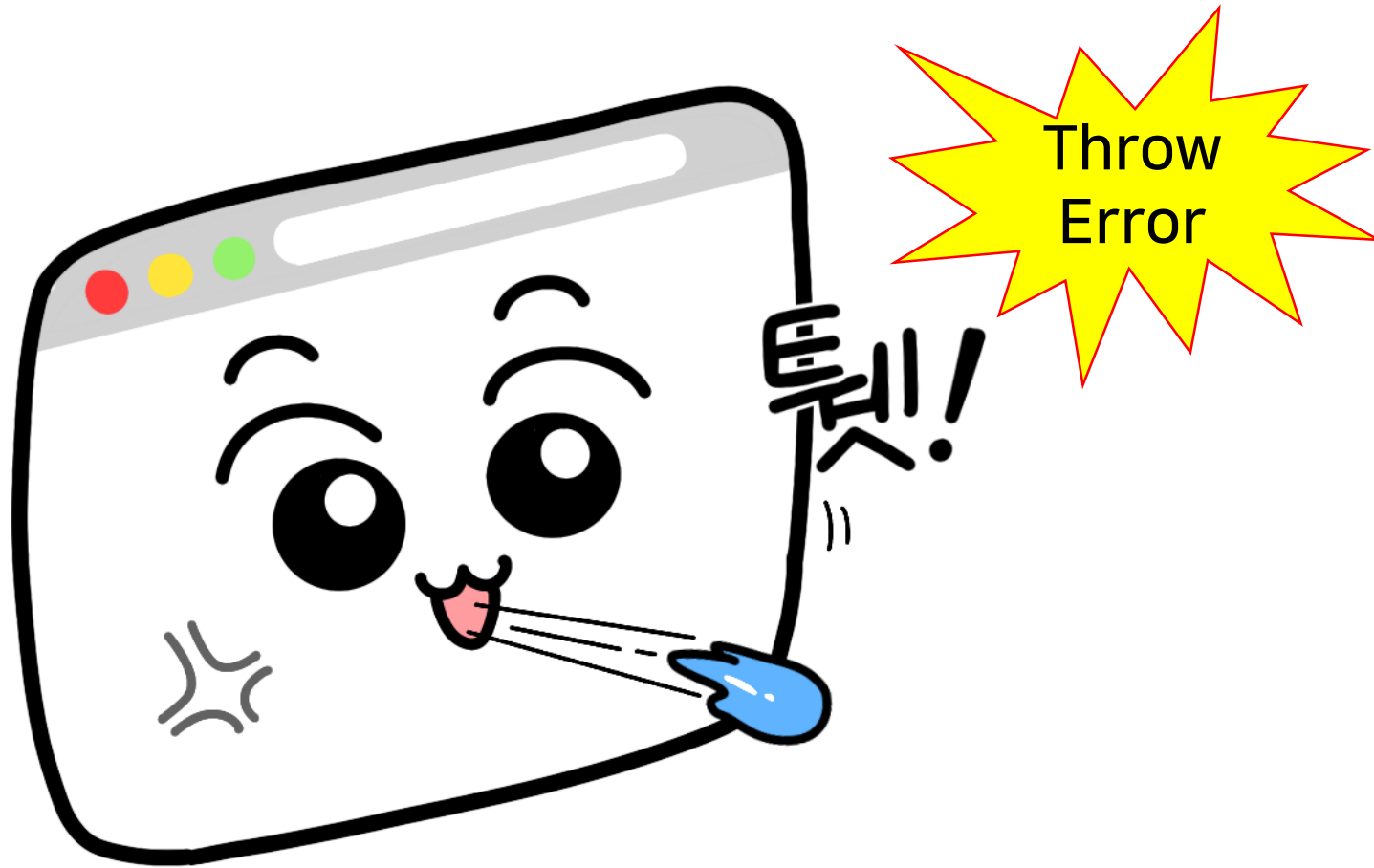
Error: 망했어요...

```
at Timeout._onTimeout (d:\git\KB_FE\Vue\chapter02\03_try_catch.js:8:11)
```

```
at listOnTimeout (node:internal/timers:569:17)
```

```
at process.processTimers (node:internal/timers:512:7)
```

에러가 발생해도
프로그램이 유지 됩니다! 😊



Throw
Error

ERROR

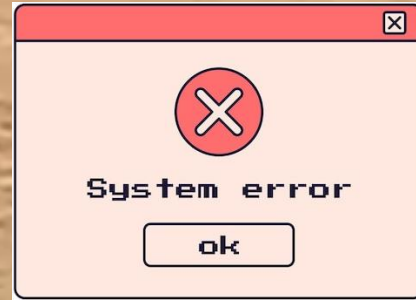
CATCH







- 안 돼! 못가 !! 가려거든 나 밟고 지나가!!



앗, 이런!

이 웹페이지를 표시하는 도중 문제가 발생했습니다.

오류 코드: Out of Memory

[자세히 알아보기](#)

[새로고침](#)





실제적으로



お題 by [コブラ・I](#) photo by [コブラ・I](#)

오랜만에 만난 지인의 이름이
생각나지 않는다.



```
<body>
  <h1 class="header">Hello, Protocol</h1>
  <button onclick="fetchData()">백엔드 통신 경험하기</button>
  <button onclick="errorHandling()">백엔드 에러 경험하기</button>
</body>
<script>
  const headerEl = document.querySelector('.header');
  const fetchData = () => {
    fetch('http://localhost:4000/request')
      .then((res) => {
        return res.json();
      })
      .then((data) => {
        console.log(data);
        headerEl.innerHTML = data;
      });
  };
  async function errorHandling() {
    try {
      const res = await fetch('http://localhost:4000/error');
      const data = await res.json();
      console.log(data);
      headerEl.innerHTML = data;
    } catch (err) {
      console.log(err);
    }
  }
</script>
```

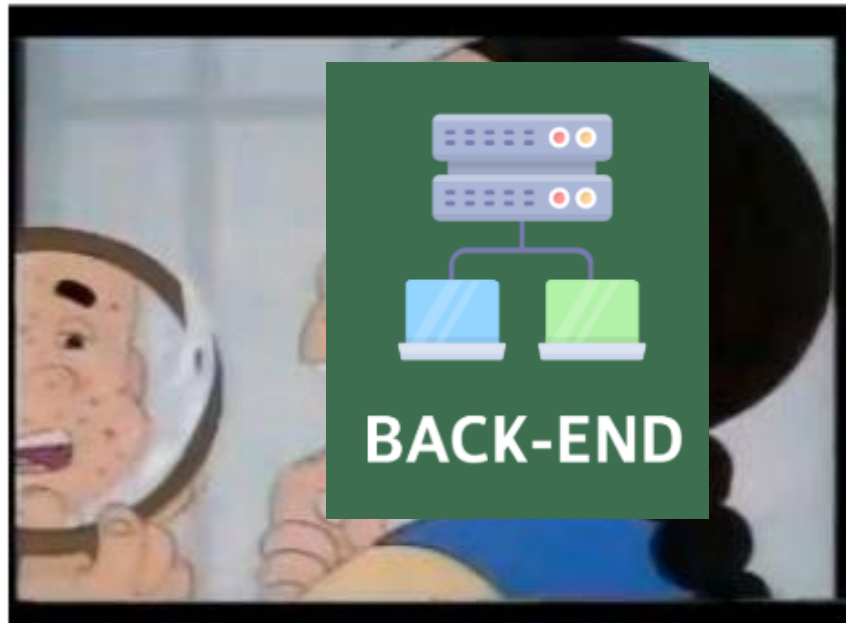
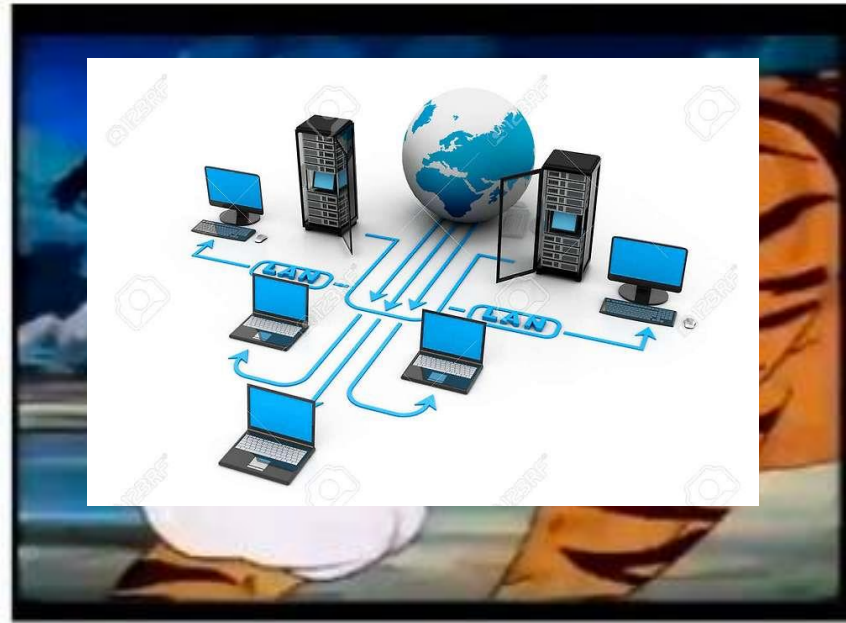
이전에 쓴 적이 있습니다!

기억 나시는 분!? 😊

```
async function errorHandling() {  
  try {  
    const res = await fetch('http://localhost:4000/error');  
    const data = await res.json();  
    console.log(data);  
    headerEl.innerHTML = data;  
  } catch (err) {  
    console.log(err);  
  }  
}
```



이러한 데이터 통신 파트에서
발생 가능한 에러는 어떤 것들이 있을까요?



마음이 든든하다



실습, try/catch 로 비동기 통신 처리



- <https://port-0-tetz-night-back-m5yo5gmx92cc34bc.sel4.cloudtype.app/test>
- 주소로 GET 요청을 보내면 아래와 같은 JSON 객체가 주어집니다!
- 해당 데이터를 받아와서 h1 태그의 내용을 변경하는 코드를 완성해 주세요.
- 단, async/await 와 try/catch 를 사용 하셔야만 합니다!

```
{  
  "message" : "여기는 Tetz 백엔드 서버 입니다"  
}
```



```
<!DOCTYPE html>
<html lang="en">
  <body>
    <h1 class="header">Hello, Protocol</h1>
    <button onclick="fetchData()">백엔드 통신 경험하기</button>
  </body>
  <script>
    const headerEl = document.querySelector('.header');
    const endPoint =
      'https://port-0-tetz-night-back-m5yo5gmx92cc34bc.sel4.cloudtype.app/test';

    // 아래의 부분을 완성해 주세요!
  </script>
</html>
```



Array Func



배열 처리에 대해서 배워 봅시다~!

- 배열은 프로그래밍에서 데이터를 처리하기위해 정말 많이 사용 됩니다!
- 그렇다면 간단한 예제를 풀어 봅시다~!

실습, 조건을 만족하는 배열의 index 구하기



- 매개변수로 배열과, 특정 수강생의 이름을 받아서 해당 이름을 가지는 배열의 index 값을 리턴하는 findIndex 함수를 작성하세요.
 - 단, 조건을 만족하는 배열을 못 찾았을 경우에는 -1 을 리턴해야 합니다!
- 배열을 순회하면서 각각의 객체에 condition: 'good' 이라는 키와 값을 추가하는 addProperty 함수를 작성하세요.

```
const classArr = [
  { id: 1, name: '안은현' },
  { id: 2, name: '강병현' },
  { id: 3, name: '염다빈' },
  { id: 4, name: '권다연' },
  { id: 5, name: '이혜원' },
  { id: 6, name: '김은지' },
  { id: 7, name: '이준범' },
  { id: 8, name: '김지민' },
  { id: 9, name: '김연후' },
  { id: 10, name: '강수민' },
  { id: 11, name: '김보성' },
  { id: 12, name: '양서진' },
  { id: 13, name: '장현지' },
  { id: 14, name: '최재원' },
  { id: 15, name: '채수정' },
  { id: 16, name: '유예원' },
  { id: 17, name: '최예빈' },
  { id: 18, name: '전규진' },
  { id: 19, name: '강민재' },
  { id: 20, name: '김어진' },
  { id: 21, name: '김영오' },
  { id: 22, name: '서승준' },
  { id: 23, name: '이호진' },
  { id: 24, name: '김희주' },
  { id: 25, name: '곽효재' },
  { id: 26, name: '유수상' },
  { id: 27, name: '김서연' },
  { id: 28, name: '안태현' },
];
```





배열 처리에 대해서 배워 봅시다~!

- 배열은 지금까지 배운 것처럼 for 문으로 대부분의 문제를 해결 할 수 있습니다!
- 다만, 코드가 길어지고 복잡해지는 환경에서 배열에 대한 처리를 해줄 때마다 for 문을 쓰면 해당 for 문이 어떤 작업을 하는지 코드를 전부 읽어줘야 하기 때문에 가독성이 떨어집니다
- 이런 문제를 해결하고자 배열은 다양한 내장 함수를 제공합니다!



findIndex

arr.findIndex



- 아까 직접 구현했던 findIndex 함수와 동일한 기능을 하는 내장 함수를 사용해 봅시다!



```
const findIndex = classArr.findIndex(function (student, index) {  
  console.log(student, index);  
  return student.name === '서승준';  
});
```

배열에 findIndex
내장 함수 사용

return 에 index 를
찾고 싶은 조건 입력하기

콜백 함수의 매개변수로
배열의 각각의 값과
배열의 index 값 받아오기



```
const findIndex2 = classArr.findIndex((student, index) => student.name === '서승준');
```

배열에 `findIndex`
내장 함수 사용

콜백 함수의 매개변수로
배열의 각각의 값을 받아오기

화살표 함수이므로
코드가 한 줄 + 리턴 값 일 경우
{ 중괄호 } 와 `return` 생략 가능



```
const findedIndex = classArr.findIndex(function (student, index) {  
  console.log(student, index);  
  return student.name === '서승준';  
});  
  
const findedIndex2 = classArr.findIndex(  
  (student, index) => student.name === '서승준'  
);  
  
console.log(findedIndex, findedIndex2);
```

```
{ id: 21, name: '김영오' } 20  
{ id: 22, name: '서승준' } 21  
21 21
```

내부에서 반복을 수행하다가
return 에 부합하는 배열을 찾으면
거기서 반복이 종료!



배열 함수의 기본

- 다른 배열 함수를 사용해도 이러한 개념은 동일 합니다!

```
classArr.배열함수(function (각각의배열, 배열인덱스) {  
    return 원하는조건;  
});  
  
classArr.배열함수((각각의배열, 배열인덱스) => 원하는조건);
```

- 매개 변수에는 각각의 배열 값이 들어온다!
- return 에 원하는 조건을 넘겨주면 된다!



filter

arr.filter



- 특정 조건을 만족하는 배열만을 골라서 새로운 배열을 만들어 리턴하는 함수입니다!
- 그럼, 이번에는 id 가 10 이상인 수강생만 존재하는 배열을 만들어 봅시다!



```
aClassArr.배열함수(function (각각의배열, 배열인덱스) { return 원하는조건; })
```

- 위의 기본 내용을 상기하면서 아래의 조건에 맞게 입력해 봅시다!
- 일단 배열 함수는 filter
- 조건은 각각의 배열(=수강생)의 id 가 10 이상입니다!



```
classArr.배열함수(function (각각의배열, 배열인덱스) { return 원하는조건; })
```

```
const overIdTenArr = classArr.filter(function (student) {  
  return student.id >= 10;  
});
```

```
console.log('🚀 ~ overIdTenArr ~ overIdTenArr:', overIdTenArr);
```

```
🚀 ~ overIdTenArr ~ overIdTenArr: [  
  { id: 10, name: '강수민' },  
  { id: 11, name: '김보성' },  
  { id: 12, name: '양서진' },  
  { id: 13, name: '장현지' },  
  { id: 14, name: '최재원' },  
  { id: 15, name: '채수정' },  
  { id: 16, name: '유예원' },  
  { id: 17, name: '최예빈' },  
  { id: 18, name: '전규진' },  
  { id: 19, name: '강민재' },  
  { id: 20, name: '김어진' },  
  { id: 21, name: '김영오' },  
  { id: 22, name: '서승준' },  
  { id: 23, name: '이호진' },  
  { id: 24, name: '김희주' },  
  { id: 25, name: '곽효재' },  
  { id: 26, name: '유수상' },  
  { id: 27, name: '김서연' },  
  { id: 28, name: '안태현' }]
```



실습, 조건을 만족하는 배열 만들기

- id 가 홀수인 수강생만을 가지는 배열을 filter 를 이용해 구하세요. 단, 콜백을 일반 함수로 사용하세요.
- id 가 짝수인 수강생만을 가지는 배열을 filter 를 이용해 구하세요. 단, 콜백을 화살표 함수로 사용하세요.



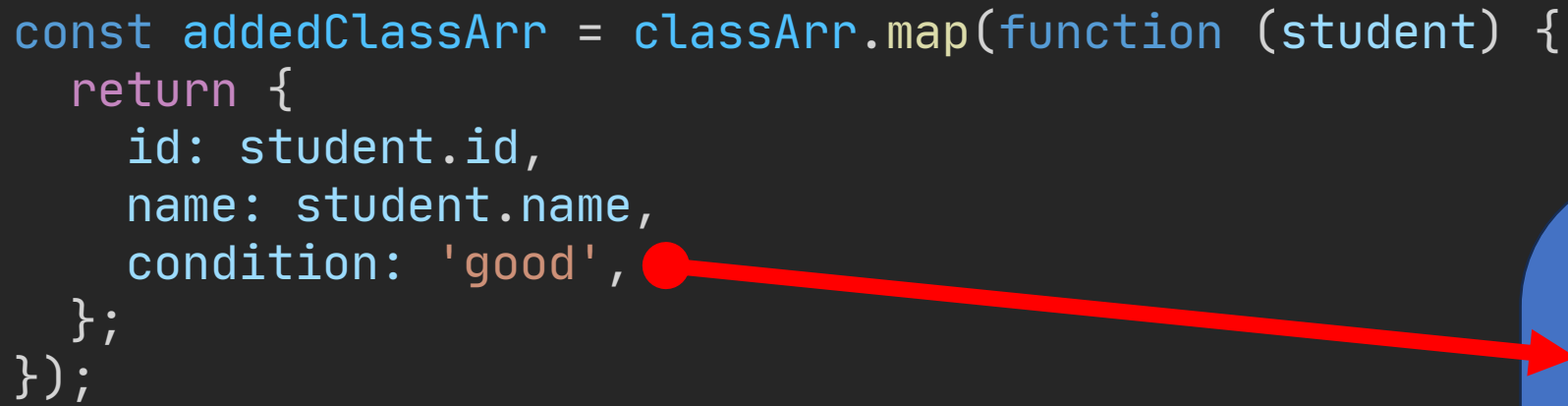
map



arr.map

- 각각의 배열 전부에 원하는 작업을 해서 새로운 배열을 만들 수 있는 함수
- return 으로 부여한 값으로 새로운 배열을 만들어 냅니다!
- 아까 각각의 배열에 condition: 'good' 을 추가했던 addProperty 함수 실습을 기억하시나요?
- 해당 작업을 map 으로 해봅시다!

```
aClassArr.map(function (각각의배열, 배열인덱스) {  
    return 원하는배열의값;  
});
```



기존의 배열을 이용해서
새로운 배열을 만들 수
있습니다!

즉, 기존 배열에
특정 작업을 추가한
배열을 새롭게 만들 수 있죠!

```
~ addedClassArr ~ addedClassArr: [
  { id: 1, name: '안은현', condition: 'good' },
  { id: 2, name: '강병현', condition: 'good' },
  { id: 3, name: '염다빈', condition: 'good' },
  { id: 4, name: '권다연', condition: 'good' },
  { id: 5, name: '이혜원', condition: 'good' },
  { id: 6, name: '김은지', condition: 'good' },
  { id: 7, name: '이준범', condition: 'good' },
  { id: 8, name: '김지민', condition: 'good' },
  { id: 9, name: '김연후', condition: 'good' },
```

그런데 말입니다...



만약 객체의 프로퍼티가
1000000 개 라면!?

1000000 개를 전부
다 써줘야 할까요!?



장점 계속 써요? ...한 400가지 되는데

전개 연산자 등장!

전개 연산자





```
const addedClassArr = classArr.map(function (student) {  
  return {  
    ...student,  
    condition: 'good',  
  };  
});
```

전개 연산자를 사용하여
기존 객체의 프로퍼티를
하나하나 뿌려서 새로운
객체에 넣습니다

```
const addedClassArr = classArr.map(function (student) {  
  return {  
    id: student.id,  
    name: student.name,  
    condition: 'good',  
  };  
});
```



실습, 아래의 조건을 만족하는 배열 만들기



- id 가 홀수인 수강생은 id 의 값을 $x2 + 1$ 하고, id 가 짝수인 수강생은 수강생의 이름 뒤에 '(짝수번)' 을 추가한 새로운 배열 editedClassArr 을 만드세요
- ex) [{ id: 3, name: '안은현' }, { id: 2, name: '강병현(짝수번)' } ...]



reduce



arr.reduce

- 배열에 대한 기본적인 기능 + 무언가 합계를 저장할 수 있는 '누산기' 를 제공
- 한 줄 코더들의 빛과 소금 reduce 입니다!
- 배열에 관련된 거의 모든 작업을 reduce 로 구현이 가능하기 때문이죠!
- 일단, 각각의 목적에 맞는 함수를 쓰는 것을 추천 드리며 reduce 는 숙달이 되신 이후에 쓰시는 것을 추천 드립니다!



```
classArr.reduce(function (합계, 각각의배열, 배열인덱스) {  
    return 합계에대한처리;  
}, 합계초기값);
```

- 위의 기본 내용을 상기하면서 아래의 조건에 맞게 입력해 봅시다!
- 일단 배열 함수는 reduce
- 조건은 각각의 수강생의 id 값을 합계에 더하시면 됩니다!
- 합계초기값은 0 으로 해주시면 됩니다! 안할 경우 undefined!

```
const sumId = classArr.reduce(function (acc, cur, index) {  
  return (acc += cur.id);  
}, 0);
```

```
console.log('🚀 ~ sumId ~ sumId:', sumId);
```



0 으로 초기화된 합계 값에
cur 에 들어오는 각각의 배열의
id 값을 더해주면 됩니다!

```
🚀 ~ sumId ~ sumId: 406
```

누산기에 합산 된 값이
리턴되어 id 의 총합이 출력

실습, 모든 수강생 이름을 합쳐봅시다!



- 모든 수강생의 이름을 하나로 합쳐서 긴~~~~~ 문자열을 만들어 봅시다!
- ex) '안은현강병현~~~~김서연안태현'